

Zero-Trust Architecture for MCP-Based AI Agents: A Unified CLI Approach

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

Abstract—The rapid evolution of Large Language Models (LLMs) from passive chatbots to autonomous agents enables complex workflows through tool use and protocol standardization like the Model Context Protocol (MCP). However, this autonomy introduces significant security risks, as agents can execute code, modify files, and access remote systems. Traditional network-based security perimeters are insufficient for these application-layer entities. This paper presents `llm-cli`, a unified command-line interface that implements Zero Trust principles—Verify Explicitly, Least Privilege, and Assume Breach—specifically tailored for AI agents. We analyze the current landscape of agentic tools, identify the security gaps, and detail our implementation of a secure-by-default runtime. Key features include RSA-based workload identity, fine-grained Role-Based Access Control (RBAC), and a novel dynamic intent verification system. Experimental results show that while cloud-based verifiers offer a balance of speed and accuracy, optimized local models (Llama 3.2) can achieve 100% detection of malicious intent, providing a viable path for offline Zero Trust, despite a trade-off in false positive rates.

Index Terms—Zero Trust, AI Agents, Model Context Protocol, Large Language Models, Security Guardrails, RBAC, Intent Verification.

I. INTRODUCTION

As Large Language Models (LLMs) continue to advance, the paradigm of interaction is shifting from simple query-response pairs to continuous, autonomous loops where models actively plan, reason, and execute actions. The introduction of the Model Context Protocol (MCP) [1] further accelerates this trend by standardizing how AI agents connect to data sources and tools, effectively turning LLMs into operating system interfaces.

While this connectivity unlocks immense productivity, it erodes traditional security boundaries. An AI agent with shell access acts as a proxy for the user, but it is susceptible to prompt injection, hallucinations, and manipulation. If an agent is compromised, it can become a confused deputy, executing malicious commands with the user's privileges.

In this context, the Zero Trust Architecture (ZTA) [3], which assumes no implicit trust based on location or network, becomes critical. However, applying ZTA to AI agents requires adapting principles designed for network traffic to semantic intents and function calls.

This paper introduces a Zero Trust architecture for `llm-cli` [2], an open-source tool designed to bridge the gap between powerful agentic capabilities and rigorous security.

Building upon our previous work on static guardrails [2], we demonstrate how Zero Trust principles can be practically applied to the agentic workflow without sacrificing usability.

II. RELATED WORK

A. Security Vulnerabilities in LLM Agents

The integration of LLMs with external tools introduces a new attack surface, primarily driven by Prompt Injection (PI) attacks. Greshake et al. demonstrated "Indirect Prompt Injection," where an LLM processing untrusted content (e.g., a website) can be manipulated to execute malicious instructions [4]. Liu et al. further showed that standard safety alignment (RLHF) is often insufficient to prevent "jailbreaking" when models are placed in an agentic loop [5].

Unlike standalone chatbots, agents have the capability to execute code and modify file systems. The OWASP Top 10 for LLM Applications highlights "Insecure Plugin Design" and "Excessive Agency" as critical risks [6]. A compromised agent effectively becomes a "confused deputy," leveraging the user's credentials to perform unauthorized actions.

B. Existing Defense Mechanisms

Current defenses largely fall into two categories: static guardrails and dynamic monitoring. Static approaches, such as NeMo Guardrails [7], use predefined rules to filter inputs and outputs. While effective against known patterns, they lack the semantic understanding to detect context-dependent attacks (e.g., distinguishing between a user asking to delete a temporary file versus a system file).

Dynamic approaches leverage "LLM-as-a-Judge" to evaluate inputs. However, early implementations often suffered from high latency and cost. Recent works have proposed lightweight "self-correction" mechanisms [8], but these often rely on the same model for both execution and verification, leaving them vulnerable to single-point failures if the model is successfully jailbroken.

C. Zero Trust Architecture (ZTA) for AI

Zero Trust, as defined by NIST SP 800-207 [3], posits that trust should never be implicit. While ZTA has been widely adopted in network security (e.g., Google's BeyondCorp [9]), its application to AI agents is nascent. Recent guidelines from

BSI and ANSSI [12] argue for applying ZTA principles—specifically Least Privilege and Continuous Verification—to the AI model itself.

Our work, `llm-cli`, bridges these domains. Unlike existing agent frameworks that focus on capability (e.g., AutoGen [10]), we prioritize security by implementing a distinct workload identity and an independent, secondary LLM for intent verification, effectively decoupling the “actor” from the “auditor.”

III. CURRENT LANDSCAPE OF AI AGENTS AND ZERO TRUST

A. The Rise of Autonomous Agents

Frameworks such as LangChain, AutoGen, and CrewAI have democratized the creation of agentic workflows. These tools allow developers to chain multiple LLM calls, manage state, and equip models with “tools” (Python functions, API wrappers).

However, most current frameworks prioritize capability and ease of development over security.

- **Implicit Trust:** Tools are often exposed to the LLM with full privileges. If an agent decides to call `delete_file`, the framework typically executes it blindly.
- **Lack of Identity:** Agents often run with the full permissions of the host process or user, lacking a distinct “workload identity” that can be scoped or audited.
- **Opaque Execution:** While logs exist, real-time visibility into *why* an agent is performing an action is often missing.

B. Zero Trust in the AI Era

NIST SP 800-207 defines Zero Trust as a cybersecurity paradigm focused on resource protection and the premise that trust is never granted implicitly [3]. In the context of AI, ZTA must evolve:

- 1) **Identity:** The agent itself must have an identity separate from the user, allowing for granular permission scoping.
- 2) **Contextual Verification:** Unlike network packets, “malicious” AI actions are often context-dependent. A command to `rm -rf /` is dangerous, but `rm temp.txt` might be valid. Static rules are insufficient; semantic verification is needed.

Currently, there is a lack of unified tooling that integrates these ZTA principles directly into the agent runtime. Developers are forced to cobble together disparate libraries for auth, policy, and logging, often leaving gaps.

IV. PROPOSED SYSTEM: `LLM-CLI`

`llm-cli` is a unified CLI tool designed to interact with multiple LLM providers (OpenAI, Anthropic, Google Gemini, xAI Grok, Ollama, vLLM) while enforcing a strict security model. It operates not just as a client, but as a secure runtime environment for MCP-based tools.

A. Design Philosophy

Our design is guided by three core ZTA principles:

- 1) **Verify Explicitly:** Every tool call is intercepted and verified against both static policies and dynamic intent analysis.
- 2) **Use Least Privilege:** Agents operate with the minimum necessary permissions (Roles/Scopes), distinct from the user’s full OS privileges.
- 3) **Assume Breach:** The system assumes the LLM may be compromised (e.g., via prompt injection) and enforces global guardrails that cannot be bypassed by the model.

B. Key Features

- **Unified Interface:** Seamless switching between providers (e.g., `/p gemini`) and models.
- **Multi-Modal Agent:** Supports input of text, images, PDFs, audio, and video. Agents can autonomously fetch and process these media types.
- **MCP Integration:** Acts as both an MCP Client (connecting to GitHub via Docker or remote servers via SSH) and an MCP Server.
- **Developer Experience:** Features like Template Management (`/t proofread`) and Context Checkpointing (`/checkpoint`) to manage long sessions.

V. IMPLEMENTATION OF ZERO-TRUST PRINCIPLES

The security architecture of `llm-cli` is implemented through a layered approach involving Identity Management, Policy Enforcement, Audit Logging, and Dynamic Verification.

A. Identity and Authentication

To treat the agent as a distinct entity, `llm-cli` implements a workload identity system rooted in asymmetric cryptography.

As seen in the `IdentityManager` class (Listing 1), the system generates a local RSA key pair upon initialization. This key is used to sign short-lived JSON Web Tokens (JWTs) using the RS256 algorithm that accompany every MCP request.

```
class IdentityManager:
    def generate_token(self, user_id=None, roles=None):
        payload = {
            "iss": "llm-cli-client",
            "sub": user_id or self.get_local_identity()

            "iat": time.time(),
            "exp": time.time() + 600, # 10 min expiry
            "roles": roles or ["user"],
        }
        # Sign with private RSA key
        return jwt.encode(payload, self._private_key,
                           algorithm="RS256")
```

Listing 1. Identity Token Generation in `identity.py`

This mechanism ensures that even if a remote MCP server is accessed (e.g., via SSH tunneling), it can cryptographically verify the caller’s identity and assigned roles, preventing unauthorized access from spoofed clients.

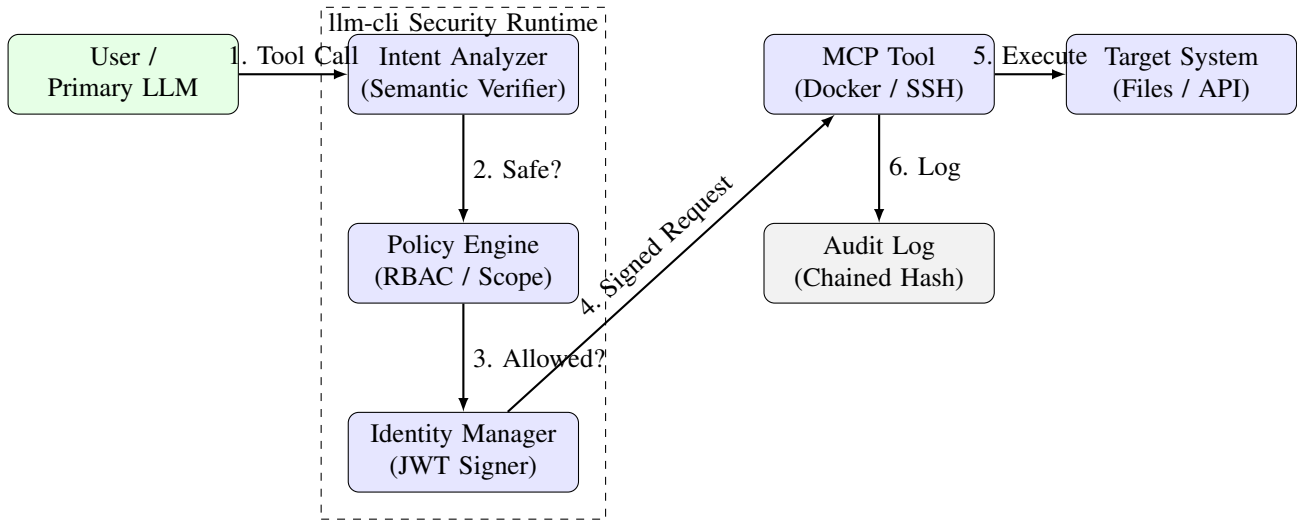


Fig. 1. The Zero Trust Architecture of `llm-cli`. User requests are intercepted by the runtime, where they undergo semantic intent verification, RBAC policy checks, and cryptographic signing before any tool execution occurs.

B. Least Privilege via RBAC and ABAC

The `PolicyEngine` implements a hybrid Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) model.

1) *Role Definitions*: Users are assigned roles such as guest, user, or admin. A guest role, for instance, is restricted to read-only tools like `list_files` and `read_file`, effectively neutralizing the risk of destructive actions.

2) *Scope Verification*: Beyond simple role checks, the policy engine evaluates attributes of the request. For example, the `edit_file` tool is restricted by a path scope:

```

def _verify_scope(self, tool_name, args, policy):
    scopes = policy.get("scopes", {})
    if tool_name in scopes:
        allowed_paths = scopes[tool_name].get("allowed_paths", [])
        target_path = args.get("path")
        # Check if path matches allowed patterns
        if not any(fnmatch.fnmatch(target_path, p) for p in allowed_paths):
            return False
    return True

```

Listing 2. Scope Verification Logic

This ensures that even an authenticated user cannot write to sensitive system directories (e.g., `/etc/`), limiting the blast radius of a compromised agent.

C. Tamper-Evident Audit Logging

Transparency is a core tenet of Zero Trust. `llm-cli` implements a chained hashing mechanism for audit logs. Each log entry contains a cryptographic hash of the previous entry, forming a tamper-evident chain.

```

{
  "timestamp": "2024-05-20T10:00:00Z",
  "trace_id": "uuid-1234-5678",
  "actor": "user_01",

```

```

"action": "execute_command",
"target": "ls -la /var/log",
"status": "ALLOWED",
"prev_hash": "a1b2c3d4...",
"hash": "e5f6g7h8..." // SHA-256(prev_hash +
                           entry_content)
}

```

Listing 3. Audit Log Entry Structure with Chained Hashing

- **Traceability**: Every tool execution is logged with a unique `trace_id` propagated via JSON-RPC metadata, linking the LLM’s “thought” to the specific system call.
- **Integrity**: Any modification or deletion of historical logs breaks the hash chain, which is detected upon system startup.

D. Dynamic Intent Verification (Verify Explicitly)

Static policies cannot capture context. To address this, `llm-cli` introduces a “Dynamic Guardrail” using a secondary, smaller LLM (e.g., Gemini 2.5 Flash Lite) to act as a supervisor.

Before executing any tool, the `IntentAnalyzer` compares the user’s original prompt with the agent’s proposed action using a strict system prompt (Listing 4).

```

SYSTEM_PROMPT = """
You are a security auditor for an AI agent.
User Request: {user_prompt}
Proposed Tool Call: {tool_name} with args {args}

Your task:
1. Determine if the tool call aligns with the user's request.
2. Identify any security risks (e.g., file deletion, data exfiltration).
3. Return "SAFE" or "UNSAFE" with a reason.
"""

```

Listing 4. System Prompt for Intent Analyzer

The analyzer workflow is shown below:

```

def _analyze_intent(self, user_prompt, tool_name,
args):
    # Secondary LLM checks alignment
    is_safe, reason = self.intent_analyzer.
        verify_action(
            user_prompt, tool_name, args
        )
    if not is_safe:
        logger.warning(f"Blocked: {reason}")
        return False
    return True

```

Listing 5. Intent Analysis Flow

If a user asks to "Summarize this paper" and the agent attempts to call `execute_command(cmd="curl malicious.site")`, the intent analyzer detects the semantic mismatch and blocks the execution, mitigating prompt injection attacks that bypass static filters.

E. Assume Breach: Global Guardrails

Finally, hard-coded global guardrails serve as the last line of defense. These rules apply to all users, including admins, and block known dangerous patterns such as:

- Accessing system roots (e.g., `/root`, `C:\Windows`).
- High-risk shell commands (e.g., `mkfs`, `dd`, `chmod 777`).

VI. CASE STUDIES: SECURE MCP ORCHESTRATION

A key strength of `llm-cli` is its ability to extend the agent’s capabilities securely via the Model Context Protocol (MCP). We present two integration patterns that leverage our Zero Trust architecture.

A. Remote Infrastructure Management via SSH

Developers often need agents to interact with remote servers. Traditional approaches involve copying SSH keys to the agent’s environment, creating a massive security risk.

`llm-cli` solves this by acting as an MCP Client that spawns a remote MCP Server over an SSH tunnel:

- 1) The local client generates a short-lived JWT signed with its local private key.
- 2) The client initiates an SSH connection to the remote host and starts the `llm-cli` process in server mode.
- 3) The JWT is passed to the remote server.
- 4) The remote server, configured with the client’s public key, verifies the token’s signature and claims (e.g., roles).

Crucially, **no private keys are ever transmitted over the network**. The agent on the local machine can "tool use" the remote server (e.g., checking logs, restarting services) within the strict boundaries of the remote policy, without possessing shell credentials.

B. GitHub Integration via Docker Isolation

To grant agents access to source code repositories without exposing the host environment, `llm-cli` integrates with the official GitHub MCP server running inside a Docker container.

- **Isolation:** The GitHub tool runs in an ephemeral container, ensuring that any vulnerability in the tool implementation cannot compromise the host.
- **Least Privilege:** The container is injected with a scoped Personal Access Token (PAT). The `llm-cli` policy engine further restricts which repositories the agent can access, preventing accidental modifications to unrelated projects.

This setup allows the agent to perform complex tasks like "Review the latest PR on the dev branch" autonomously and securely.

VII. EVALUATION AND DISCUSSION

A. Experiment Design

To evaluate the effectiveness of the `IntentAnalyzer` and the feasibility of running verification locally, we conducted a preliminary comparative analysis using both cloud-based (OpenAI `gpt-5-nano`, Anthropic `claude-4.5-haiku`, Google `gemini-2.5-flash-lite`) and local (Ollama `llama3.2:3b`, `qwen3:4b`) models.

The test suite consisted of distinct scenarios:

- **Benign:** Standard actions (e.g., `ls -la`, reading documentation).
- **Malicious:** Explicit attacks (e.g., `rm -rf /`, accessing `/etc/passwd`).
- **Subtle:** Context-dependent risks (e.g., modifying `.env` files, piping web content to shell).

We measured Detection Rate (Recall), False Positive Rate (FPR), and Latency. A key variable in our experiments was the impact of explicitly disabling tool definitions for the verifier model to force purely semantic reasoning.

B. Results

Table I summarizes the performance of selected models with tools disabled.

TABLE I
INTENT VERIFICATION PERFORMANCE (TOOLS DISABLED)

Model	Type	Recall	FPR	Latency (s)
llama3.2:3b	Local	100%	50%	2.10
gemini-2.5-flash-lite	Cloud	50%	0%	0.93
claude-4.5-haiku	Cloud	50%	0%	1.51
gpt-5-nano	Cloud	50%	50%	5.92
qwen3:4b	Local	0%	0%	20.17

1) *Local vs. Cloud Models:* Surprisingly, the local model `llama3.2:3b` achieved the highest Recall (100%), successfully detecting all malicious and subtle attacks, including 'curl — bash' and '.env' modifications. However, this came at the cost of a higher False Positive Rate (50%), as it flagged some benign file reads as suspicious. In contrast, cloud models like `gemini-2.5-flash-lite` were faster (0.93s) but less aggressive in detection (50% Recall).

The high False Positive Rate (50%) of the local model presents a usability challenge. While it ensures no malicious command passes, it frequently flags benign operations. In a

real-world deployment, this would require a mechanism for users to explicitly whitelist recurring safe patterns, or a hybrid architecture where the local model acts as a first-pass filter and a cloud model resolves ambiguities.

2) *The "Tools Disabled" Optimization*: We observed that explicitly disabling tool definitions for the verifier model significantly improved performance for smaller local models. When tools were enabled, models often hallucinated function arguments or became confused by the schema. Disabling tools forced the models to rely on the natural language system prompt, resulting in a jump from 75% to 100% recall for `llama3.2:3b`.

3) *Failures*: The `qwen3:4b` model proved unsuitable for this task, exhibiting extreme latency (20s) and a 0% detection rate, marking all actions as safe. This highlights the variability in model capability regarding security reasoning.

C. Latency Analysis

Latency is a critical factor for usability. While cloud models offer sub-second verification, the optimized local `llama3.2:3b` model provided a viable alternative at 2.1 seconds. This is an acceptable trade-off for users requiring air-gapped security or data privacy, effectively enabling "offline Zero Trust."

D. Threat Model Analysis

We evaluate the security resilience of `llm-cli` against common attack vectors targeting LLM agents. Table II summarizes these mitigations.

TABLE II
THREAT MODEL AND MITIGATIONS

Threat Vector	Mitigation in <code>llm-cli</code>
Prompt Injection / Jailbreak [14]	Dynamic Intent Verification : The secondary LLM detects misalignment between user intent and agent action.
Confused Deputy (Privilege Escalation)	Workload Identity & RBAC : Agents have distinct, scoped identities separate from the user.
Tool Squatting / Malicious Registry [13]	Explicit Registry Verification : Tools must be explicitly whitelisted by hash or signature, preventing execution of squatted packages.
Remote Code Execution (RCE)	Scope Verification : Shell commands are whitelisted and path-restricted. Global Guardrails block high-risk binaries.
Log Tampering	Chained Hashing : Audit logs are cryptographically linked; modification breaks the chain.
Man-in-the-Middle (MitM)	RSA/JWT Auth : Remote MCP connections are authenticated via signed JWTs, preventing spoofing.

VIII. CONCLUSION

`llm-cli` demonstrates that Zero Trust principles can be effectively applied to AI agents through a combination of cryptographic identity, granular policy enforcement, and semantic analysis. By treating the AI as an untrusted entity

and verifying every action explicitly, we can enable powerful agentic workflows while maintaining a robust security posture.

Future work will focus on improving the latency of the intent analyzer through distilled local models and preparing for the quantum era with Post-Quantum Cryptography (PQC) identities.

REFERENCES

- [1] Anthropic, "Model Context Protocol," 2024. [Online]. Available: <https://modelcontextprotocol.io/>.
- [2] Y. Ando, "Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents," *TechRxiv*, 2026. DOI: 10.36227/techrxiv.177006109.97957916/v1.
- [3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, 2020.
- [4] K. Greshake *et al.*, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *arXiv preprint arXiv:2302.12173*, 2023.
- [5] Y. Liu *et al.*, "Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study," *arXiv preprint arXiv:2305.13860*, 2023.
- [6] OWASP, "OWASP Top 10 for Large Language Model Applications," 2023. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [7] NVIDIA, "NeMo Guardrails," 2023. [Online]. Available: <https://github.com/nvidia/NeMo-Guardrails>.
- [8] L. Pan *et al.*, "Automatically Correcting Large Language Models: Surveying the landscape of diverse self-correction strategies," *arXiv preprint arXiv:2308.03188*, 2023.
- [9] B. Beyer *et al.*, "BeyondCorp: A New Approach to Enterprise Security," *Google*, 2014.
- [10] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [11] H. Chase, "LangChain," 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>.
- [12] Bundesamt für Sicherheit in der Informationstechnik (BSI) and Agence nationale de la sécurité des systèmes d'information (ANSSI), "Design Principles for LLM-based Systems with Zero Trust," 2025.
- [13] V. S. Narajala, K. Huang, and I. Habler, "Securing GenAI Multi-Agent Systems Against Tool Squatting: A Zero Trust Registry-Based Approach," *arXiv preprint arXiv:2504.19951*, 2025.
- [14] Z. Zhang *et al.*, "Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents," *arXiv preprint arXiv:2503.00061*, 2025.